KU LEUVEN

FACULTY OF
ENGINEERING SCIENCE

The best master's thesis ever

First Author
Second Author

Thesis submitted for the degree of
Master of Science in
Electrical Engineering, option
Electronics and Chip Design

Supervisor
Prof. dr. ir. Knows Better

Assessors
Ir. Kn. Owsmuch
K. Nowsrest

Assistant-supervisors
Ir. An Assistent
A. Friend

Academic year 2025 – 2026

Published by First Author and Second Author,
Department of Electrical Engineering, Kasteelpark Arenberg 10 bus 2440, B-3001 Leuven

Preface

This is my acknowledgment to thank everyone who has kept me busy. I would like to thank my supervisor, my advisor, and the entire jury. My family has also been very supportive, of course.

First Author
Second Author

Contents

Abstract

This `abstract` environment provides a summary of the work, which may or may not be extensive. The intention is that this should be limited to 1 page.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

List of Figures and Tables

List of Figures

List of Tables

List of abbreviations and symbols

abbreviations

LoG	Laplacian-of-Gaussian
MSE	Mean Square error
PSNR	Peak Signal-to-Noise ratio

Symbols

42	"The Answer to the Ultimate Question of Life, the Universe, and Everything" volgens de [2]
c	Speed of light
E	Energy
m	Mass
π	The number pi

Chapter 1

Introduction

Modern industrial environments are becoming increasingly connected. Systems that were once largely isolated now communicate with enterprise networks, remote services, cloud platforms and security appliances [15]. This integration improves visibility, automation and operational efficiency, but it also increases the exposure of Operational Technology (OT) environments to cyber threats [3]. Unlike traditional Information Technology (IT) systems, attacks against OT environments may have consequences beyond data loss, including disruption of physical processes, equipment damage and safety risks [28, 15].

These characteristics make security monitoring in OT environments both important and challenging. Industrial networks often contain legacy devices, strict availability requirements and communication patterns that differ strongly between sites [1, 15]. As a result, security mechanisms that are common in IT environments cannot always be transferred directly to OT environments [28]. Passive network-based monitoring is therefore an attractive approach, because it can observe communication behavior without modifying the industrial devices themselves [23].

Anomaly detection is a promising technique in this context, since it can learn a reference of normal behavior and identify deviations from that reference. This is especially relevant when labeled attack data is limited or when the kind of attacks are unknown beforehand [15, 19]. However, the practical value of anomaly detection depends strongly on the data source, the feature representation, the thresholding strategy and the evaluation setting [11]. Models that perform well on clean or controlled datasets may behave differently when applied to noisy operational data from real environments [6, 20].

This thesis was carried out in collaboration with "AXS Guard (by Appraoch Cyber)", whose security appliances are deployed at the network edge of customer environments. Because these appliances already operate as routers and firewalls, they naturally observe the connections passing through them. This makes Linux connection tracking metadata, or conntrack metadata, a practical data source for anomaly detection, as it provides lightweight summaries of observed network connections without requiring changes to OT devices or relying on deep packet inspection, which can introduce additional processing overhead, privacy concerns and deploy-

ment complexity, especially when traffic is encrypted or difficult to parse reliably. The choice for conntrack metadata is therefore motivated not only by its technical content, but also by its deployability in real customer environments.

In this thesis, this metadata is transformed into window-based feature vectors and used to evaluate whether existing unsupervised anomaly detection models can detect early-stage reconnaissance behavior. Reconnaissance is considered because it often precedes more intrusive attack phases, such as exploitation or lateral movement [3]. During this phase, an attacker attempts to discover active hosts, reachable services and communication paths in order to identify possible entry points or targets. Detecting these activities early can give defenders time to investigate or respond before the attacker progresses toward actions that may disrupt the physical process. Since reconnaissance mainly affects communication patterns, it can leave traces in network metadata even before the physical process itself is affected [29, 4].

The goal of this thesis is not to design a new anomaly detection algorithm, but to implement and evaluate existing unsupervised models as part of a consistent detection pipeline. The pipeline covers the transformation of raw conntrack events into flow records, the aggregation of these records into time windows, the extraction and selection of features, model training on benign data, threshold selection and evaluation on operational data with simulated reconnaissance attacks.

The central research question of this thesis is formulated as follows:

> *How can existing unsupervised anomaly detection models be adapted, implemented as a pipeline, and evaluated using Linux connection tracking metadata to detect early-stage reconnaissance in noisy, real-world OT environments?*

To answer this question, a pipeline is constructed where the following steps are carried out:

- reconstruct raw Linux conntrack events into connection-level records that can be used for network traffic analysis
- aggregate the reconstructed connections into time-based windows per source host, so that each window can be represented as a feature vector
- define and select conntrack-derived features that capture behaviours relevant to reconnaissance, such as destination spread, short-lived communication and asymmetric or failed connections
- simulate early-stage reconnaissance scenarios and inject them into noisy operational OT traffic to create an evaluation setting
-
- apply and configure existing unsupervised anomaly detection models, including k-NN, K-Means and autoencoder-based approaches, to the extracted conntrack-based feature vectors

- compare the models under a consistent experimental pipeline, considering the effect of design choices such as window length, feature set, threshold selection and evaluation metric.

The remainder of this thesis is structured as follows. Chapter **??** discusses related work on OT security, anomaly detection, reconnaissance, unsupervised models and evaluation metrics. Chapter **??** describes the data source, preprocessing steps and construction of the experimental datasets. Chapter **??** presents the detection pipeline and experimental setup. Chapter **??** reports and discusses the results. Finally, Chapter **??** summarizes the main findings, limitations and future work.

Chapter 2

Related Work

2.1 Operational Technology Networks and Security Challenges

Operational Technology (OT) refers to the hardware and software systems utilized for the direct monitoring, control, and actuation of physical devices, industrial processes, and events [16] They include components such as sensors, actuators, Programmable Logic Controllers (PLCs), Remote Terminal Units (RTUs), Supervisory Control and Data Acquisition (SCADA) systems, Human-Machine Interfaces (HMIs), and engineering workstations. These systems form the basis of many Cyber-Physical Systems (CPS) and are used in critical infrastructure such as power grids, water treatment facilities, transportation networks and manufacturing plants [28].

Historically, OT networks operated as isolated, air-gapped environments that relied almost entirely on physical separation for security [15]. This separation has changed with the adoption of Industry 4.0, an industrial paradigm that describes the digital transformation of manufacturing and other industrial sectors through connected, automated and data-driven systems. The Industrial Internet of Things (IIoT) is part of this shift and involves the connection of industrial sensors, devices and machines to networks for data exchange, remote monitoring and automation. As a result, OT environments are increasingly integrated with traditional IT networks. While this IT-OT convergence improves data collection, automation and operational efficiency, it also expands the cyber-attack surface. Industrial assets that were once assumed to be isolated may become reachable from corporate networks, external services, or indirectly through compromised IT systems [16, 15].

Securing these environments is difficult because IT and OT networks do not always have the same operational priorities. In traditional IT security, these priorities are often described using the CIA triad: Confidentiality, Integrity and Availability. Confidentiality protects information from unauthorized access, Integrity protects data and systems from unauthorized modification, and Availability ensures that systems remain accessible when needed. In traditional IT environmens, confidentiality often has the highest priority, followed by integraity and lastly availability. In OT environments, however, this ordering is reversed towards an AIC perspective,

with availability being the main concern. This is because OT systems are directly connected to physical processes and are often subject to real-time and safety constraints. Downtime, delayed communication, or incorrect control actions can lead not only to data loss or service disruption, but also to production loss, equipment damage, environmental impact or risks to human safety [1, 16].

These risks have already been demonstrated by several well-known cyber-physical attacks. Stuxnet targeted Siemens PLCs used in an Iranian nuclear facility and is often considered one of the first examples of malware causing direct physical damage in an industrial process. It modified the control logic of uranium-enrichment centrifuges so that they operated outside safe conditions, while simultaneously hiding this abnormal behavior from monitoring systems by reporting normal-looking values [16, 4]. BlackEnergy and Industroyer later showed how attacks against power-grid environments can move beyond compromise of IT systems and directly affect electricity distribution. In the Ukrainian power-grid incidents, attackers interacted with industrial control systems and protocols to disrupt substations and cause real power outages [16, 4]. Triton/Trisis targeted Schneider Electric Triconex Safety Instrumented Systems in a petrochemical plant. This was especially concerning because safety instrumented systems are designed to keep industrial processes within safe operating limits, meaning that the attack targeted not only production systems but also the mechanisms intended to prevent catastrophic failures [16, 28]. More recent ransomware incidents, such as EKANS and Colonial Pipeline, further show that cyber incidents do not need to directly manipulate PLC logic to affect industrial operations. EKANS included functionality to terminate industrial control processes, while the Colonial Pipeline incident led to a multi-day shutdown of fuel pipeline operations [16, 8]. These examples illustrate why OT security has to consider cyber-physical consequences, not only traditional information security risks.

Furthermore, the deployment of security solutions in OT is often limited by legacy systems and resource constraints. Many OT environments still rely on older devices and industrial protocols, such as Modbus, DNP3 and EtherNet/IP, which were designed mainly for reliable communication and not necessarily with built-in security mechanisms such as encryption or strong authentication [28, 16]. Updating or replacing such systems can be operationally risky, expensive, or delayed because of uptime requirements [1, 16]. In addition, endpoint devices at the lower levels of an ICS architecture, such as sensors, actuators and PLCs, often have limited memory and computational resources [1, 15]. This makes it difficult to deploy heavy host-based security mechanisms directly on these devices.

For these reasons, it is important to explore security approaches that are adapted to OT environments. Any monitoring or detection method has to take into account the operational constraints of industrial systems, where availability, safety and continuity are central requirements. At the same time, the method should be deployable without major changes to legacy devices or interruptions of the physical process. This is a difficult balance. OT environments need better visibility into suspicious behavior, but the security mechanism itself should not become a source of operational risk. The next section therefore discusses anomaly detection as a possible

approach for detecting abnormal behavior in OT networks.

2.2 Anomaly Detection in OT Environments

Given the constraints discussed in Section 2.1, OT security mechanisms should provide visibility into suspicious behavior without disrupting the underlying physical process. Intrusion Detection Systems (IDS) are one way to provide this visibility and are commonly divided into signature-based and anomaly-based approaches. Signature-based IDS compare observed traffic or system events against predefined rules or patterns, known as signatures, that describe previously identified malicious activity. This makes them effective for attacks that have already been characterized, but less suitable for novel or targeted attacks whose signatures are not yet availabl [19, 15] This limitation is especially relevant in OT environments, where attacks may be highly system-specific and where generating signatures for complex process or network behavior can be difficult [28].

Anomaly-based detection takes a different approach: instead of searching for known attack patterns, normal behavior is modeled based on observed data, and deviations from this model are flagged as suspicious [1]. Anomaly detection has been studied extensively in IT security, but methods developed for general IT traffic cannot simply be assumed to transfer directly to OT environments. IT traffic is often shaped by human activity, web applications, cloud services and changing user behavior, making it diverse and highly variable. OT traffic, in contrast, is more closely tied to the physical process and is often driven by fixed interactions between devices, periodic control tasks, polling behavior and stable asset roles. This makes anomaly detection attractive for OT networks, where unknown attacks are an important concern and where communication is often more regular than in general IT networks. This periodic nature of OT networks, means deviations from expected communication can provide more useful indicators of abnormal behavior. [19, 6].

However, this regularity does not mean that a single generic baseline can be applied to all OT networks. Jeffrey et al. [15] describe IT networks as being closer to a “monoculture”, with many similar systems and more standardized communication protocols, while OT networks are much more heterogeneous and differ strongly between organizations. A pattern that is normal in one industrial environment may therefore be unusual in another. For this reason, OT anomaly detection cannot simply reuse IT-based methods or generic traffic models, but requires environment-specific baselines that capture the temporal and asset-level behavior of the monitored operational context [15].

A major challenge in developing machine learning-based anomaly detection for OT environments is the limited availability of labeled attack data. Cyberattacks on critical infrastructure are rare, difficult to observe, and unsafe to reproduce in operational environments. As a result, real-world OT datasets are often dominated by benign traffic, while labeled malicious examples are scarce or unavailable [15, 28]. This makes fully supervised learning difficult, since such models require representative examples of both normal and attack behavior. Even when attack labels are

available in public datasets, they may not cover the full range of realistic attacks or environment-specific behavior. For this reason, one-class and semi-supervised approaches are frequently used in OT anomaly detection [6, 15, 19]. In this setting, the model is trained mainly or exclusively on benign data and learns a representation of normal operation. During detection, samples that deviate sufficiently from this learned normal behavior receive a high anomaly score.

2.3 Limitations of Existing OT Anomaly Detection

Although one-class and semi-supervised anomaly detection methods are a practical direction for OT networks, existing research still has several limitations that make it difficult to transfer results to real operational environments. A first limitation is the strong reliance on a small number of public benchmark datasets, such as SWaT, WADI and BATADAL [12, 1]. These datasets are valuable because they provide labeled attack scenarios and allow different methods to be tested on a common baseline. However, they are usually collected in controlled test environments and therefore do not fully represent the conditions found in real OT networks. Operational environments can contain noisy traffic, benign misconfigurations, maintenance activity, incomplete labels or customer-specific behavior that are difficult to capture in benchmark datasets [20, 15]. As a result, models that perform well on clean and controlled datasets may not generalize directly to real, noisy environments.

Another limitation is that results across OT anomaly detection studies are difficult to compare directly. Existing work evaluates a wide range of methods, from distance-based and clustering approaches to tree-based outlier detectors and deep reconstruction-based models, but these are often tested with different datasets, preprocessing steps, feature sets, thresholding strategies and model configurations [28, 12]. As a result, it is difficult to determine whether a reported improvement is caused by the model architecture itself, by the data pipeline, or by the evaluation setup. If models cannot be compared under consistent conditions, it is also difficult to decide which approach is actually suitable for a specific OT monitoring problem. Recent evaluations further show that preprocessing choices and detection hyperparameters can strongly influence the final result, while the specific model architecture is not always the dominant factor [12]. Hence comparing candidate models within the same data pipeline and evaluation setup leads to more useful conclusions rather than treating reported performance numbers from different studies as directly comparable.

A final limitation of much OT anomaly detection research is that it focuses mainly on physical process variables, such as sensor readings, actuator states, tank levels or other process measurements. This is natural for datasets such as SWaT and WADI, where attacks are often represented through changes in the physical process. However, it also means that many proposed methods are evaluated on attacks that have already affected the process itself [12, 3]. Less attention is given to earlier network-level activity, such as host discovery, port scanning or other reconnaissance behaviour, even though these actions may occur before direct manipulation of the

physical infrastructure.

2.4 Early-Stage Detection and Reconnaissance

The focus on physical process variables can be understood through the Cyber Kill Chain (CKC), which as depicted in Figure 2.1, describes an intrusion as a sequence of stages: reconnaissance, weaponization, delivery, exploitation, installation, command and control, and action on objectives [3]. In the early stages, the attacker gathers information about the target environment, prepares or delivers the attack, and attempts to gain a foothold. In the later stages, the attacker establishes persistence, communicates with compromised systems, and ultimately executes their objective. In an OT context, this final objective may involve manipulating the physical process, for example through false data or command injection [3]. Process-level anomaly detection mainly observes these later stages, where the attacker has already reached the point of affecting the physical infrastructure. While such detection is still important, it leaves less time for operators to react before operational consequences occur.

For this reason, early-stage detection has become an important direction for OT security. The initial stages of an intrusion can last much longer than the final execution phase, providing defenders with more time to detect suspicious behavior and respond before the attack reaches its objective [3]

Reconnaissance is especially relevant in this context because it is often one of the first steps performed by an attacker. Before exploiting a target, the attacker typically tries to understand the network by identifying active hosts, open ports, exposed services and possible entry points. This can involve host discovery, port scanning and service enumeration [17, 4]. These activities cannot be observed through physical processes but leave observable traces in network communication, such as connections to many destinations, probes to many ports, short-lived TCP flows or failed connection attempts. Detecting such patterns can provide an early warning before the attacker moves further along the kill chain [29, 26].

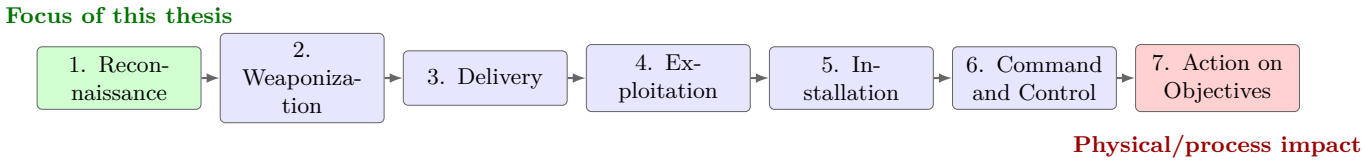


FIGURE 2.1: Cyber Kill Chain stages.

2.5 Unsupervised Models for Network-Based Anomaly Detection

To detect the early-stage reconnaissance activities discussed in Section 2.4, multiple unsupervised and semi-unsupervised models were explored that can identify

deviations in network traffic without relying on known attack signatures.

Distance-based algorithms, and especially k-Nearest Neighbours (k-NN), are frequently used as lightweight anomaly detection models [19, 15, 5]. In a one-class setting, k-NN stores benign reference samples and scores a new observation according to its distance from the closest benign samples. Meira et al. [19] evaluated several unsupervised techniques for cyber-attack anomaly detection on the public NSL-KDD and ISCX network intrusion datasets, including One-Class Nearest Neighbour, One-Class K-Means, Isolation Forest, One-Class SVM and an autoencoder. Their results show that nearest-neighbour methods can perform competitively on network intrusion data, especially on the ISCX dataset [19]. Jeffrey et al. also motivate the use of one-class models in CPS environments, arguing that malicious data is often scarce and that modelling benign behaviour directly is more suitable than relying on balanced two-class training data [15]. In their experiments, one-class k-NN achieved the best accuracy among the evaluated one-class and two-class models, which supports its use as a simple but strong baseline [15].

A related line of work combines distance-based detection with statistical descriptions of network behavior. Entropy-based features are particularly relevant for network anomaly detection because they summarize how concentrated or dispersed communication is across hosts, ports or protocols. The ADAM scheme proposed by Cai et al. [5] uses entropy vectors as traffic profiles and combines them with unsupervised anomaly detection for Distributed Denial of Service (DDoS) mitigation in software-defined cyber-physical systems. In particular, ADAM selects k-NN as the anomaly detection module because it provides a practical balance between detection effectiveness and the runtime overhead of classifying new entropy vectors. Although ADAM targets DDoS attacks rather than reconnaissance, it is relevant for this thesis because reconnaissance can also change the distribution of contacted destinations and ports. This motivates the inclusion of entropy-based feature groups when evaluating distance-based models.

Clustering approaches have also shown promising results in industrial anomaly detection. For one-class anomaly detection, K-Means can represent benign behavior as a set of clusters, after which new samples are scored by their distance to the nearest cluster centroid. Wang et al. [28] compare several supervised and unsupervised models on the SWaT testbed and report that K-Means performs strongly among the unsupervised methods, achieving the highest recall and F1-score among the unsupervised techniques and one of the lowest undetected rates. In this context, recall measures how many attacks are detected, while F1-score balances detection recall with false alarms. This suggests that relatively simple clustering models can capture useful structure in deterministic OT environments. However, this result is still based on a controlled benchmark testbed. It therefore remains useful to evaluate whether the same type of simple clustering model can remain effective on noisier operational network metadata.

Alongside distance-based and clustering methods, reconstruction-based deep learning models are widely used in CPS and OT anomaly detection. Autoencoders are trained to reconstruct their input after compressing it through a lower-dimensional representation. When trained on benign data, the underlying assumption is that

normal samples will be reconstructed well, while anomalous samples will result in a higher reconstruction error. Ortega-Fernandez et al. [23] propose a deep-autoencoder-based network intrusion detection system for DDoS attacks in ICS using network flow data. Their work is especially relevant because it focuses on flow-based features rather than device logs or process variables, requires limited preprocessing and evaluates the feasibility of the approach on data of a real industrial plant. However, for safety reasons, attacks could not be executed in the production environment, so the real-world evaluation mainly concerns false alarms and deployment feasibility, while attack detection is evaluated on a separate ICS DDoS dataset.

A practical challenge for autoencoder-based methods is that benign training data is not always perfectly clean. Operational data may contain rare benign behavior, measurement noise, imperfect labels or hidden outliers. Catillo et al. [6] address this issue in CPS-GUARD, which combines a semi-supervised autoencoder with an outlier-aware threshold selection strategy based on Isolation Forest. With a focus on practical deployability, they argue that CPS intrusion detection should favor semi-supervised learning, non-intrusive data sources and simple model designs over unnecessarily complex architectures.

Finally, recent evaluations suggest that increasing model complexity does not automatically improve anomaly detection performance. Fung et al. perform a comprehensive evaluation of reconstruction-based anomaly detection in ICS and show that model architecture and size are not always the dominant factors, with smaller models often performing similarly to larger architectures [11]. This supports the use of simple models as meaningful baselines rather than treating them only as weak alternatives to complex deep learning models.

2.6 Evaluation Metrics

The choice of evaluation metric is an important part of anomaly detection research, especially in OT environments where attacks are rare compared to normal operation. When malign data is sparse, overall accuracy can be misleading, as a model that predicts almost everything as benign may still obtain a high accuracy while failing to detect the relevant attacks [28, 11]. For this reason, anomaly detection studies commonly report precision, recall and F1-score, which provide a more informative view of the trade-off between false positives and false negatives.

However, standard precision, recall and F1-score are usually computed point-wise, meaning that each timestamp or window is evaluated independently. This is not always aligned with the way attacks appear in practice. Many OT and network attacks unfold over a temporal interval rather than as isolated samples. A reconnaissance scan, for example, may generate a sequence of anomalous windows, while a low-and-slow scan may be spread over a much longer period. Point-wise metrics do not explicitly distinguish between detecting an attack immediately and detecting it near the end of the attack interval. They can also give disproportionate weight to long attacks, because every anomalous window contributes separately to the final

score [11].

To address this, recent work has proposed the use of range-based metrics for temporal anomaly detection. Instead of evaluating each window independently, range-based metrics group consecutive anomalous windows into temporal ranges and compare predicted anomaly ranges with the true attack ranges [11]. This better matches operational alert analysis, where the main concern is whether an attack episode is detected, how fragmented the alerts are, and whether detection occurs early enough to support response. This distinction matters because the metric used for tuning and evaluation can change which model appears best. Fung et al. [11] show that evaluating ICS anomaly detectors with range-based metrics can lead to different conclusions than using point-F1 alone.

2.7 Thesis Positioning

The reviewed literature motivates three main choices in this thesis. First, the detection problem is studied using operational network metadata rather than clean benchmark data or physical process variables. The input consists of Linux conntrack metadata collected from AXS Guard appliances, which provides lightweight visibility into communication behaviour at the network edge without relying on packet payloads or host-based agents.

Second, the thesis focuses on early-stage reconnaissance instead of later-stage physical process manipulation. Host discovery, port scanning, service detection and low-and-slow scanning are treated as deviations from the normal communication behaviour of a source host. This places the work earlier in the Cyber Kill Chain, where detection can provide more time for investigation before the physical process is affected.

Third, the thesis evaluates existing unsupervised model families under a common experimental setup. Instead of proposing a new model architecture, it compares lightweight distance- and clustering-based methods with a reconstruction-based autoencoder approach using the same windowing, feature extraction, thresholding and evaluation pipeline. This makes it possible to study whether models that perform well in the literature remain useful on noisy operational OT metadata.

The following chapters therefore examine how conntrack flows are reconstructed, how reconnaissance attacks are simulated and injected into real benign traffic, and how window-level features and anomaly detection models are evaluated using both point-based and range-based metrics.

Chapter 3

Dataset Construction and Experimental Setup

This chapter describes how the datasets used in the experiments were constructed. The process starts from raw Linux conntrack metadata collected from real OT environments. These binary event files are reconstructed into unidirectional flow records, and split chronologically into training, validation, and test periods. Attack data is simulated and remapped to match the companies' data context. The resulting output is therefore a set of flow-level datasets that serve as input to the feature extraction and detection methodology described in Chapter 4.

3.1 Data Source and Scope

3.1.1 Linux Connection Tracking Metadata

This thesis was carried out in collaboration with AXS Guard, a Belgian cybersecurity company based in Mechelen [13]. AXS Guard develops and manages cybersecurity solutions for organizations of all types, with a focus on security gateways and managed cybersecurity services. They provided the real network metadata used in this thesis, which originates from operational customer environments and therefore represents realistic OT communication patterns.

The metadata was collected from AXS Guard appliances deployed at the network edge, where they operate as routers and firewalls. This deployment context is important for the design of the dataset. The appliances must perform their primary networking and security functions continuously, while operating under hardware constraints that depend on the specific model. Some appliances have limited memory resources, starting from 1 GB on the least performant models, and going up to 12 GB or 16 GB of memory [14]. These constraints motivate the use of lightweight network metadata rather than packet payloads or host-level process information.

For this reason, the dataset is based on Linux conntrack metadata. Conntrack is the connection-tracking subsystem of the Linux kernel and is part of the Netfilter framework. It collects metadata about observed network flows, such as source and

destination addresses, ports, protocol, packet and byte counters, timestamps, and connection state. Conntrack is commonly used by firewalling and NAT systems such as iptables and nftables, but in this thesis it is used only as a source of lightweight network-flow metadata for anomaly detection [21, 25]. Since conntrack is already natively integrated into the Linux networking stack, it allows the system to passively extract useful network visibility without requiring complex network reconfiguration or additional heavyweight monitoring components.

The dataset should therefore be interpreted as network connection metadata observed at the monitored subnet or gateway. It does not provide a complete view of all communication in the customer organization, but only of the traffic visible from the selected observation point. It also does not contain packet payloads, sensor readings, actuator states, PLC register values, or other process-level variables. Instead, the analysis in this thesis is based on metadata describing communication patterns between endpoints.

3.1.2 Data Confidentiality & Limitations

The datasets used come from real OT environments of the collaborating companies. Because these environments are operationally sensitive, several details are intentionally omitted or anonymized. This includes the names of the companies, exact network topologies, device roles, raw IP addresses, and any information that could reveal the structure or operation of the industrial networks. Where examples are shown, synthetic or anonymized values are used.

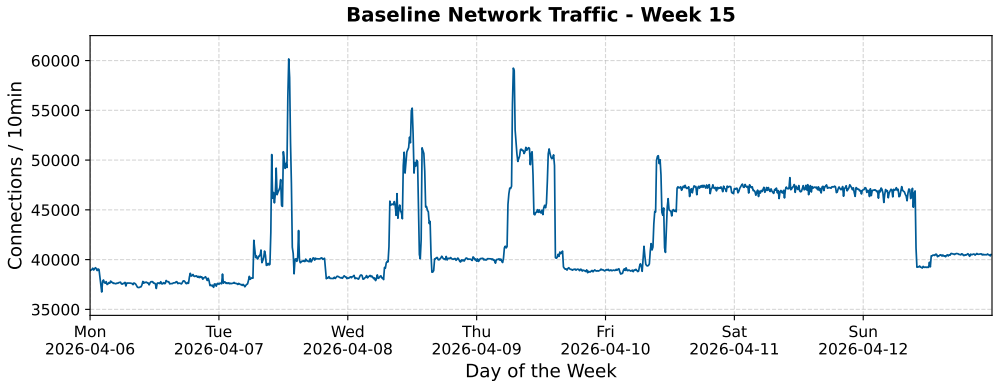
For safety and operational reasons, reconnaissance attacks were not executed directly against the OT environments. Even harmless scanning activity can disrupt fragile or legacy industrial devices, trigger alerts, or interfere with normal operations. Instead, reconnaissance behavior was simulated in a controlled environment and injected into the real benign traffic. This approach allows the evaluation of attack detection methods while avoiding unnecessary risk to customer systems.

3.1.3 Data Scope

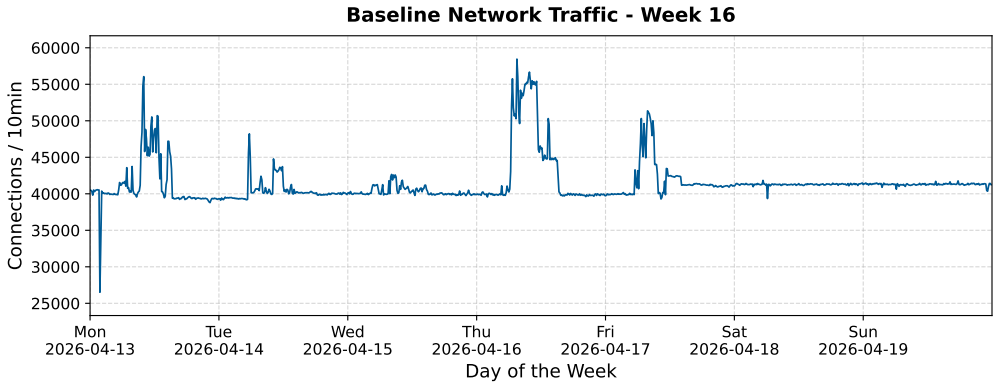
The OT dataset consists of metadata from two real environments, which was collected during the months of March and April of 2026. The collected traffic represents normal operational communication observed during the measurement periods. No confirmed "attacks" were present in the collected company data, however some abnormalities may be present due to noise or misconfigurations.

Figure 3.1 shows two weeks of traffic from one of the companies, represented as the number of connections per 10-minute interval. In week 16, the traffic pattern is consistent with what would be expected from an OT environment. The number of connections follows a periodic structure, with higher and more spiky activity during working hours and lower activity after hours, during the weekend. In week 15, however, the traffic pattern is more irregular. Between 2026-04-10 and 2026-04-13, there is a sudden higher but constant number of connections. Internal inspection linked this irregularity mainly to one source host, which generated many additional

connections and changed its usual destination pattern by communicating with other servers on port 80. This bahavior shows that OT traffic, while benign, can still contain unusual patterns.



(A) Baseline traffic for customer 1, week 15.



(B) Baseline traffic for customer 1, week 16.

FIGURE 3.1: Baseline network traffic for customer 1 across two selected weeks.

3.2 Data Pre-processing

3.2.1 Conntrack Ingestion

The raw conntrack data was logged as compressed binary event files. Each file contained a sequence of conntrack events rather than already finalized flow records. Therefore, for the first preprocessing step, the files were decompressed and and parsed into structured records.

Table 3.1 shows a small preview of the raw event stream after decompression and initial parsing. The parser processes the event stream sequentially and maintains a dictionary of active connections indexed by connection identifier. `ADD` events initialize new active connections using the available packet metadata. `UPDATE` events

Table 3.1: Preview of raw conntrack events (anonymized).

Event	Conn. ID	Timestamp	L4	Source IP	Destination IP	Orig. Bytes	Reply Bytes
LIST	105672107	2026-03-19 13:54:39.559	6	10.20.109.217	10.20.37.127	422	1421
LIST	3843433871	2026-03-19 13:54:39.559	6	10.20.109.217	10.20.235.187	422	1551
DELETE	4122098797	2026-03-19 13:54:39.582		nan	nan	422	1048
ADD	4122098797	2026-03-19 13:54:39.582	6	10.20.109.217	10.20.146.19	0	0
ADD	1297319782	2026-03-19 13:54:39.638	6	10.20.109.217	10.20.10.241	0	0
DELETE	923955930	2026-03-19 13:54:40.343		nan	nan	422	1666
DROP	28924	2026-03-19 13:54:56.343	17	10.20.85.60	10.20.144.114	64	
DROP	0	2026-03-19 13:54:56.343	6	10.20.3.179	10.20.199.224	40	
UPDATE	711927862	2026-03-19 14:04:39.553		nan	nan	462	1492

provide intermediate cumulative byte and packet counters for an active connection, while `DELETE` events finalize connections using the end timestamp and final byte and packet counters. `LIST`, `EMPTY`, `HEADER`, and `DROP` events are skipped because they do not describe a connection state transition that can be mapped to a reconstructed flow record. `HEADER` events contain stream-level metadata, `EMPTY` events contain no connection data, `LIST` events are used for listing existing conntrack entries, and `DROP` events indicate records that were discarded.

The resulting finalized connections, initially produced in connection-completion order, are converted into unidirectional flow records and sorted chronologically by start time in order to preserve information about communication directionality. Important characteristics of reconnaissance are not just the existence of a connection, but also which host initiated the communication, which hosts or ports were targeted, and whether the probing activity received responses. Table 3.2 provides an example of flow produced by this preprocessing step.

Table 3.2: Preview of reconstructed unidirectional conntrack flows (anonymized).

Start Time	End Time	L4	Src. IP	Src. Port	Dst. IP	Dst. Port	Bytes	Packets	Orig. Direction
2026-03-19 13:54:39.582	2026-03-19 13:56:43.254	6	10.10.109.217	47282	10.10.144.114	80	462	7	True
2026-03-19 13:54:39.582	2026-03-19 13:56:43.254	6	10.10.144.114	80	10.10.109.217	47282	1940	7	False
2026-03-19 13:54:39.638	2026-03-19 13:56:43.251	6	10.10.109.217	56620	10.10.10.241	80	422	6	True
2026-03-19 13:54:39.638	2026-03-19 13:56:43.251	6	10.10.10.241	80	10.10.109.217	56620	1615	6	False
2026-03-19 13:54:39.664	2026-03-19 13:56:43.224	6	10.10.146.19	80	10.10.109.217	49516	1908	7	False
2026-03-19 13:54:39.664	2026-03-19 13:56:43.224	6	10.10.109.217	49516	10.10.146.19	80	423	6	True

3.2.2 Data Splitting

After ingestion, the benign company traffic was divided into training, validation, and test periods using a 70–15–15 chronological split. The split was performed at the conntrack-flow level before feature extraction. This was done because of the usage

of multiple window sizes during model comparison. If the data were first converted into window-level features and only then split, the exact train, validation, and test periods could depend on the chosen window length. Splitting the underlying flow records first, ensures that all feature extraction runs are based on the same temporal data partitions, making results across different window sizes comparable.

The split was also performed before attack injection, as described in Section 3.3. This ensures that the training set remains free of simulated attack traffic, which is important for the semi-supervised approach of the thesis, ensuring the models will be trained on data representing purely normal behavior of the targeted environment. The validation and test splits are used as benign background traffic with only them having malicious traces injected. The validation data is used for model selection, threshold tuning, and feature/window configuration choices, while the test data is kept separate for the final performance evaluation.

3.3 Attack Simulation

3.3.1 Attack Environment

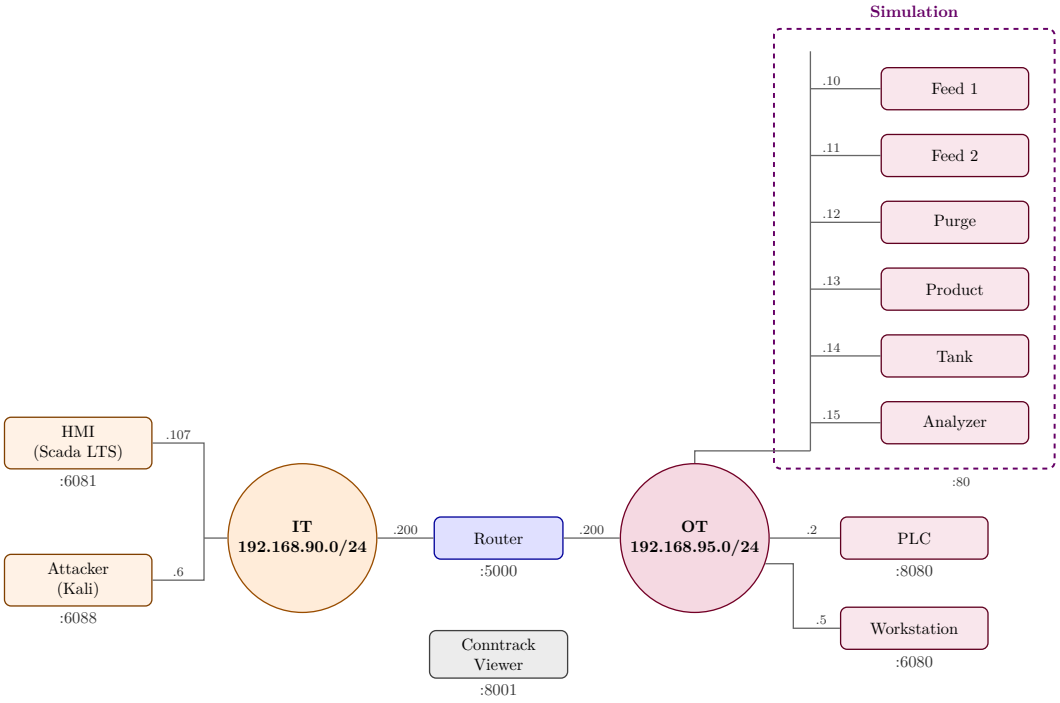


FIGURE 3.2: GRFICSv3 Network Diagram

Due to the safety and operational concerns discussed in 3.1.2, the reconnaissance attacks were not executed against the companies' OT environments. Instead, they were simulated in a virtual OT lab based on the open-source GRFICSv3 environ-

ment [9]. GRFICSv3 is a containerized ICS/OT cyber-physical security lab that simulates an industrial chemical plant.

The attacks were run from the Kali attacker container (`192.168.90.6`) which, as shown in Figure 3.2, is located in the IT network. This represents the scenario where an attacker has obtained control over an IT-side host and performs reconnaissance towards the OT environment. The router/firewall container acts as the observation point, collecting all conntrack metadata form the crossing traffic

3.3.2 Attack Scenarios

The simulated attacks were designed to represent different forms of reconnaissance. The goal was to generate traffic patterns associated with host discovery, port discovery, service probing, and low-rate scanning.

The scenarios vary in scan intensity and target scope. Some scans target the full OT subnet, while others target a smaller set of known live OT hosts. The full OT subnet corresponds to `192.168.95.0/24`. The selected live-host target set consists of known active hosts in the virtual OT network, including `192.168.95.2`, `192.168.95.5`, and `192.168.95.10-192.168.95.15`. The simulated attacks are summarized in Table 3.3.

Table 3.3: Overview of simulated reconnaissance attack types.

Attack type	Scan configuration	Target scope
Aggressive subnet scan	`nmap -sS -Pn -T5 -top-ports 100`	Full OT subnet
Fast subnet scan	`nmap -sS -Pn -T4 -top-ports 100/200`	Full OT subnet
Default-speed subnet scan	`nmap -sS -Pn -T3 -top-ports 100`	Full OT subnet
Slow subnet scan	`nmap -sS -Pn -T2 -top-ports 50/100`	Full OT subnet
Very Slow targeted scan	`nmap -sS -Pn -T1 -p selected ports`	Selected live OT hosts
Extremely slow targeted scan	`nmap -sS -Pn -T0 -p selected ports`	Selected live OT hosts
Service/version detection	`nmap -sV -Pn -p selected ports`	Selected live OT hosts
Host discovery	`nmap -sn`	Full OT subnet
Low-and-slow scan	`nmap -sS -Pn -T2 -top-ports 50`	Full OT subnet

For scanning `nmap` was selected because it is a widely used network scanning tool for host discovery, port scanning, and service identification, making it suitable for generating controlled reconnaissance traffic with configurable scan intensity and timing [22]. Most port discovery scenarios use TCP SYN scanning (`-sS`) with host discovery disabled (`-Pn`), so nmap proceeds directly to probing the specified targets. The timing templates range from very aggressive (`-T5`) to extremely slow (`-T0`), allowing the dataset to include both faster and slower reconnaissance behavior. The low-and-slow scenario is included as an Advanced Persistent Threat (APT) inspired reconnaissance pattern, where probing activity is spread over a longer and interrupted period rather than concentrated in a short scan burst.

The selected port sets include common administrative and web ports such as `22`, `80`, and `443`. They also include OT-relevant services, most notably Modbus/TCP on port `502`. Modbus/TCP is commonly used for supervision and control of automation

equipment over TCP/IP networks, and port `502` is frequently treated as the default Modbus/TCP port in ICS scanning and measurement studies [23, 4]. In some runs, port `8080` was also included to cover web-based industrial interfaces.

Additional scenarios include host discovery (`-sn`), which identifies reachable devices in the target network without performing a full port scan. Service/version detection (`-sV`) is also performed as a later reconnaissance step where the scanner attempts to identify the applications or protocols running behind exposed ports [22].

3.3.3 Attack Injection

After executing the described reconnaissance attack scenarios, the adversarial traffic was isolated from the benign simulation traffic. This isolation was based on the recorded attack timelines, the known attacker address, and the target scope of each scenario. Conntrack records were flagged as malign when their timestamps fell within the corresponding attack interval and their endpoints matched the expected bidirectional attacker-to-target communication.

The isolated attack traces could not be directly inserted into the company data, since much of their metadata would not match. Specifically, the IP addresses and the timestamps from the virtual OT lab was modified in order to blend in. The IP remapping was deterministic, ensuring that attacker-side and target-side addresses were projected consistently onto IT-side and OT-side company ranges, respectively. This preserves the intended attack direction of an IT host performing reconnaissance towards OT targets. The timestamps of the isolated attack traces were also shifted into the relevant periods of the company data. The relative timing within each attack was preserved, so that scan bursts, idle gaps, and low-and-slow patterns remained intact after injection.

After data remapping, the malicious data could be injected into the validation and testing splits. Labels were assigned during this integration step. Original company traffic was treated as benign, while the injected reconnaissance records were labeled as malign traffic. The different attack names were also retained as metadata allowing for later evaluation of results at a per attack scenario.

It should be noted that while the injected traffic provides a useful approximation of how reconnaissance attacks may appear in the different customer environments, not all effects of an attack executed directly inside the companies' networks are reproduced, such as device-specific responses, firewall behavior, or operational side effects.

Chapter 4

Detection Methodology

4.1 Windowing

The split, sorted and unidirectional labeled conntrack flows could not be used directly for model training. Instead, they were aggregated into features representing flow properties over a fixed-length time windows in order to describe the communication behavior of each source host over time.

Each model input is defined by a source IP address, a timestamp and a window length. In other words, all flows related to the same source host within that same time window are aggregated into one feature vector. For example, when using a 60-minute window, all flows associated with some source host `10.20.109.217` from 10:00 to 11:00 are combined into one vector containing features such as the number of contacted destination IPs, destination ports, protocol distribution, and packet or byte statistics. An example can be seen in Table 4.1.

TABLE 4.1: Example feature vectors after window-based aggregation.

src_ip	win_id	win_start	uniq_dst	rx_bytes	tx_pkts	mean_B/pkt	mean_dur_s
10.20.109.217	492947	1774609200000	1	164	4	151.75	167.229

Prior work has shown that host-level connection patterns are effective for traffic classification and anomaly detection, even when payloads are unavailable, traffic is unlabeled, or unidirectional[7]. Similarly, as discussed in Section 2.4, reconnaissance behaviour can leave observable traces in network communication, such as connections to many destinations, probes to many ports, short-lived TCP flows or failed connection attempts. This makes a source-host window representation suitable for detecting reconnaissance activity.

Feature extraction was performed on training, validation, and test splits for four different window sizes: 60 s, 300 s, 3600 s, and 18 000 s, corresponding to one minute, five minutes, one hour and five hours respectively. Using multiple window sizes makes it possible to compare how detection performance changes when short bursts and longer low-rate reconnaissance activity are aggregated at different temporal resolutions.

4.2 Features

4.2.1 List of Features

As discussed in Section 2.4, reconnaissance is an early-stage attack activity aimed at discovering reachable hosts, open ports, exposed services, and network structure. In network-flow metadata, this behaviour is expected to appear as a change in how a source host communicates. A scanning host may contact more destinations than usual, probe more destination ports, generate many short or low-volume connections, and receive fewer replies when targets are inactive or closed. A list of features was chosen as a representation of this type of source behavior during the specified time window, and are displayed in Table 4.2. A more detailed explanation of each feature can be found in Appendix A.1.

Table 4.2: Overview of extracted feature groups.

Feature group	Features
Window metadata	`ip_src`, `window_id`, `window_start_ts`
Volume	`flow_count`, `total_bytes`, `total_packets`, `sent_bytes`, `sent_packets`, `received_bytes`, `received_packets`
Duration and packet statistics	`mean_duration_s`, `std_duration_s`, `mean_bytes_per_packet`, `std_bytes_per_packet`, `bytes_per_flow`, `packets_per_flow`
Destination and port diversity	`unique_dst_ip`, `unique_dst_port`, `unique_src_port`, `dst_port_per_dst_ip_ratio`
Directionality and reply behavior	`outbound_ratio`, `reply_packet_ratio`, `reply_byte_ratio`, `no_reply_packet_score`, `no_reply_byte_score`, `is_pure_upload_like`, `is_pure_download_like`
Entropy	`l4_proto_entropy`, `protocol_entropy`, `ip_src_entropy_window`, `ip_dst_entropy`, `port_src_entropy`, `port_dst_entropy`, `tcp_src_port_entropy`, `tcp_dst_port_entropy`, `udp_src_port_entropy`, `udp_dst_port_entropy`
TCP scan behavior	`tcp_flow_count`, `tcp_short_flow_count`, `tcp_short_flow_rate`
Temporal change	`flow_count_delta1`, `flow_count_mean3`, `unique_dst_ip_delta1`, `unique_dst_ip_mean3`, `unique_dst_port_delta1`, `unique_dst_port_mean3`, `tcp_short_flow_rate_delta1`, `tcp_short_flow_rate_mean3`, `no_reply_packet_score_delta1`, `no_reply_packet_score_mean3`, `ip_dst_entropy_delta1`, `ip_dst_entropy_mean3`, `port_dst_entropy_delta1`, `port_dst_entropy_mean3`, `outbound_ratio_delta1`, `outbound_ratio_mean3`

The **window metadata** column identifies the source host and corresponding time window as supported by [7].

Volume features describe the amount of communication associated with a source host. They include `flow_count`, `total_bytes`, `total_packets`, `sent_bytes`, `sent_packets`, `received_bytes`, and `received_packets`. These features

capture whether a host becomes more active than usual and whether the activity is mainly outgoing, incoming, or balanced. Similar flow-based anomaly detection systems are described by Dromard [10], who also relies on sliding-window statistics to capture abrupt deviations in traffic behavior.

Flow duration and packet statistics describe another dimension of network behavior. Features such as `mean_duration_s` and `std_duration_s` summarize how long the observed flows last, while `mean_bytes_per_packet`, `std_bytes_per_packe t`, `bytes_per_flow`, and `packets_per_flow` describe how much traffic is exchanged per flow and per packet. These features are relevant for reconnaissance traffic which may produce many short and lightweight connections, especially during host or port probing. Similar features are included in standard NetFlow based feature sets for network intrusion detection [27]

The **destination and port diversity** features capture how widely a source host communicates within a time window. `unique_dst_ip` measures the number of distinct destinations contacted, while `unique_dst_port`, `unique_src_port`, and `dst_port_per_dst_ip_ratio` describe the diversity of contacted services and ports. These features are directly relevant to reconnaissance, because horizontal scans contact many hosts, vertical scans contact many ports on one host, and mixed scans may affect both dimensions [26].

The **directionality and reply-behavior** features are included to capture whether a source host mainly initiates communication and whether those attempts receive substantial responses. Features such as `outbound_ratio`, `reply_packet_ratio`, `reply_byte_ratio`, `no_reply_packet_score`, and `no_reply_byte_score` are especially useful for scan-like behavior, where probes may be sent towards closed ports, inactive hosts, filtered targets, or services that do not exist [26].

The **TCP scan behavior** features focus on TCP-specific probing. Since many of the simulated reconnaissance scenarios use TCP SYN scanning, the number and fraction of short TCP flows are included as indicators of incomplete or unsuccessful TCP connection attempts. In a SYN scan, the attacker sends the initial SYN request and typically does not complete the three-way handshake for open ports [26].

The **entropy** features summarize how concentrated or dispersed communication is. Low entropy indicates that most traffic is concentrated on a small number of values, while higher entropy indicates a more diverse distribution. For a categorical feature X with possible observed values v_i, the Shannon entropy is computed as:

$$H(X) = -\sum_{i=1}^{m} p(v_i) \log_2 p(v_i),$$

where $p(v_i)$ is the relative frequency of value v_i in the considered source-host window, and m is the number of distinct observed values. For example, `port_dst_entrop y` is computed from the distribution of destination ports contacted by a source host in a window, while `ip_dst_entropy` is computed from the distribution of contacted destination IP addresses. A host that repeatedly communicates with one service has lower destination-port entropy than a host whose traffic is spread across many ports, which can indicate scanning behavior. Cai et al. uses these exact features

in their proposed ADAM scheme, which makes up one of the tested models in this thesis as discussed in Section 4.3.1.

Finally, the **temporal change features** describe how specific behavioral features evolve across consecutive windows. The `_delta1` features measure the change relative to the previous window for the same source host, while the `_mean3` features compute a short rolling average over the current and two previous windows. For the first observed window of each source host, no previous window is available; therefore, the corresponding `_delta1` value is set to zero and the `_mean3` value is equal to the current window value. For the second window, `_mean3` is computed over the first two available windows, and from the third window onward it is computed over the current and two previous windows. The three-window average provides limited temporal smoothing without aggregating behavior over too long a period. These features are included because slow reconnaissance may not produce a strong anomaly in a single isolated window, but may become more visible over time [26].

4.2.2 Feature Sets

Feature sets were constructed using an iterative selection process, by comparing the defined features from Section 4.2.1 of the benign and malign data. For each feature, benign and malign validation windows were compared using their mean and median values. For each feature f, the benign mean μ_b, malicious mean μ_m, benign standard deviation σ_b, and malicious standard deviation σ_m were computed. To rank features in a scale-independent way, the difference between the benign and malicious means was normalized by the pooled standard deviation, resulting in a Cohen's d standardized mean difference [18].

The pooled standard deviation is defined as

$$\sigma_{\text{pooled}} = \sqrt{\frac{\sigma_b^2 + \sigma_m^2}{2}},$$

The standardized mean difference for feature f is then computed as

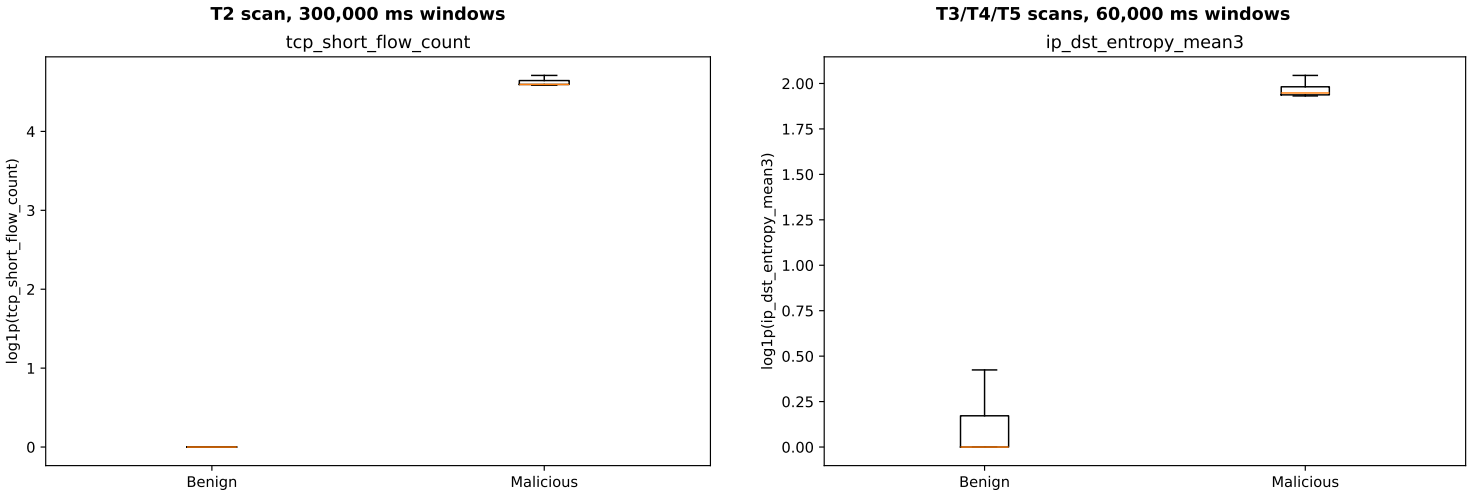
$$\text{SMD}(f) = \frac{\mu_m - \mu_b}{\sigma_{\text{pooled}}}.$$

Features were ranked by the absolute value $|\text{SMD}(f)|$, where the vertical bars indicate that only the magnitude of the standardized difference is considered. This allows features with different numerical scales to be compared fairly and ensures that features are ranked highly regardless of whether their values increase or decrease during attacks. Median differences were also inspected to make the ranking easier to interpret for possible skewed features. This comparison was repeated for each different time window (60 s, 300 s, 3600 s, and 18 000 s) and for each different attack category grouped by intensity (T0/T1, T2, T3/T4/T5). As an indicator, Table 4.3 captures a global ranking of feature importance, combining all of the different window, attack combinations.

The ranked summaries of each attack group and window were used to identify feature families that consistently distinguished reconnaissance from benign traffic.

TABLE 4.3: Global feature ranking based on the median absolute standardized mean difference across attack/window-specific rankings.

Rank	Feature	Median |SMD|	Mean |SMD|
1	ip_dst_entropy_mean3	7.046000	7.026000
2	ip_dst_entropy	6.704000	6.773000
3	tcp_short_flow_rate_mean3	6.246000	5.608000
4	tcp_short_flow_rate	6.215000	5.573000
5	unique_dst_ip_mean3	3.511000	9.212000
6	no_reply_packet_score_mean3	2.942000	3.101000
7	no_reply_packet_score	2.922000	3.088000
8	unique_dst_ip	2.801000	8.598000
9	is_pure_upload_like	2.294000	2.407000
10	outbound_ratio_mean3	2.273000	2.363000



(a) (b)

FIGURE 4.1: Benign–malicious boxplot comparisons for selected reconnaissance-related features. (a) `tcp_short_flow_count` for the T2 attack using 5-minute windows. (b) `ip_dst_entropy_mean3` for the T3/T4/T5 attacks using 1-minute windows.

Some of the strongest recurring features were `ip_dst_entropy` and `ip_dst_entropy_mean3` representing destination spread, `tcp_short_flow_rate` and `tcp_short_flow_rate_mean3` representing short-lived TCP scan behavior, and `no_reply_packet_score` and `outbound_ratio` representing failed or asymmetric communication. Two of these features are illustrated as boxplots in Figure 4.1, visualizing the separation. Based on this information, candidate feature sets were constructed to cover multiple aspects of reconnaissance behavior while limiting redundancy between strongly related features.

This resulted in three types of feature sets. First, general-purpose sets, combining the strongest non-redundant features. Second, attack specific sets emphasizing particular behavior, such as fast and broad scans, stealthy and slow scans, or horizontal host discovery. The last type, isolated specific signal families, such as entropy-only (ADAM), or asymmetry-only groups to get more information on which underlying reconnaissance behaviors contributed most to detection. It is important to clarify that the test split was not used during this feature-set construction process in order to avoid leakage. The exact feature sets can be observed in Appendix A.2.

4.3 Anomaly Detection Models

All models in this section are evaluated in a one-class anomaly detection setting. During training, the models do not rely on labeled data as they are only trained on benign windows, learning a reference of what normal traffic in a specific environment looks like and assigning an anomaly score to unseen windows based on their deviation from this benign reference. Ground-truth labels are introduced only after scoring, in order to evaluate performance.

4.3.1 k-Nearest Neighbors

As discussed in Section 2.5, k-Nearest Neighbors (k-NN) is traditionally a supervised learning algorithm, where a new sample is classified based on the labels of its closest neighbors in the training data. However, the same distance-based principle can also be used for one-class anomaly detection. There is no explicit training phase in which model parameters are learned. Instead, the benign training observations are stored as a reference set in the selected feature space. During evaluation, each new sample is compared to these stored benign observations using a distance metric. If the sample is close to previously observed normal behavior, it receives a low anomaly score. If it is far from the benign reference set, it receives a high anomaly score. A threshold is then used to decide when this distance is large enough for the window to be considered anomalous.

This principle is reflected in our implementation. The k-NN model is fitted only on the benign training split for which training data is first sorted chronologically by `window_start_ts` and `window_id`. The first 80% of the sorted benign training data is used to build the nearest-neighbor reference set, while the remaining 20% is kept aside for threshold calibration. This calibration subset is still benign, but is not used to fit the k-NN model itself. After fitting, the model is applied to the calibration

subset to estimate the range of distance scores that can still occur under normal behavior. Different percentile thresholds are then tested based on this benign score distribution.

Features are scaled using `RobustScaler` [24], which is fitted only on the 80% reference subset and then applied unchanged to the calibration and evaluation data. This scaler centers each feature around its median and scales it according to the interquartile range, meaning the spread between the 25th and 75th percentiles. Robust scaling was chosen because derived features may contain extreme values even in benign traffic, the effect of which is contained with this method. The nearest-neighbor search is implemented using the `NearestNeighbors` class from `scikit-learn` [24], configured with Euclidean distance. Meira et al. empirically validated the use of standard Euclidean distance as the core distance metric when successfully applying One-Class Nearest Neighbor algorithms to network intrusion detection datasets [19]. For each input sample, the distances to the k nearest benign reference samples is computed, by averaging the score to this k nearest neighbors.

ADAM

An additional k-NN variant was included for evaluation based on the feature representation proposed in ADAM. ADAM combines entropy-based traffic profiling with unsupervised anomaly detection for DDoS mitigation in software-defined cyber-physical systems. Normal traffic is represented using entropy based features over the network and transport layer, and it selects k-NN as the anomaly detection modules because of its balance between time overhead and detection effectiveness [5]. In order to evaluate the effectiveness of the ADAM proposed feature set, some experiments were run exclusevily using features form the Entropy group in Table 4.2.

4.3.2 Outlier-aware Denoising Autoencoder

The next anomaly detection approach is based on the the CPS-GUARD method, which uses a semi-supervised deep autoencoder together with an outlier aware threshold selection procedure [6]. Autoencoders are neural networks trained to reconstruct their own input. In anomaly detection, the underlying assumption is that the model trained on benign data will learn to reconstruct normal behavior well, while windows that differ from this learned normal behavior will result in a larger reconstruction error. This error can therefore be used as an anomaly score.

The model is again trained only on benign training windows. The selected input features are first scaled using a preprocessing pipeline consisting of `RobustScaler` followed by `MinMaxScaler` with a target range of $[-1, 1]$. Robust scaling is used to reduce the influence of extreme feature values, while the min-max scaling step matches the range of the autoencoder output layer, which uses a `tanh` activation function.

The autoencoder follows a symmetric encoder-decoder structure. Several architectures are evaluated, including the original CPS-GUARD-style N-36-6-36-N architecture but also larger or smaller variants, where N is the number of selected

input features. The architecture therefore maps an N-dimensional input vector to a first hidden layer with 36 neurons, compresses it to a 6-neuron bottleneck representation, expands it again to 36 neurons, and finally reconstructs an N-dimensional output vector. The hidden layers use ReLU activations, while the output layer uses `tanh`.The model is trained with mean squared error as reconstruction loss and RMSProp as optimizer. To reduce overfitting to the benign training windows, dropout is applied after the first two hidden layers, as well as Gaussian input noise as a regularization mechanism. A diagram of this model initialized with N-36-6-36-N architecture is depicted in Figure 4.2.

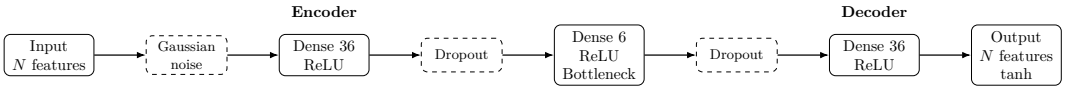


Figure 4.2: CPS-GUARD-style autoencoder architecture used for reconstruction-based anomaly detection. The N-dimensional feature vector is regularized using Gaussian noise, compressed into a lower-dimensional bottleneck representation, and then reconstructed at the output. Dropout is applied after the first two hidden layers during training. [6]

For threshold selection, the benign training data is divided into an autoencoder training subset and a separate benign threshold-calibration subset, which is excluded from training. After training, the calibration subset is passed through the autoencoder and each calibration window receives a reconstruction error, which represents a range of scores that can still occur at benign inputs. Then Isolation Forest is applied to the calibration subset to separate benign calibration samples from outliers [6]. The contamination value controls the expected fraction of calibration samples that Isolation Forest treats as outliers. Testing different contamination values therefore changes how aggressively unusual benign calibration windows are removed before the final anomaly threshold is selected. In this way, instead of directly selecting a fixed percentile over all benign reconstruction errors, possible outliers in the benign calibration set can have less influence on the threshold.

4.3.3 K-means

The final model used is based on K-means clustering. K-means is an unsupervised clustering algorithm that partitions the training data into k clusters by assigning samples to the nearest cluster centroid. Although K-means is primarily a clustering algorithm, it can be adapted to one-class anomaly detection by treating the distance to the nearest benign cluster centroid as an anomaly score [19]. Then, during evaluation, a new sample is compared to these centroids. Samples close to one of the benign clusters receive a low anomaly score, while samples far away from all learned centroids receive a high anomaly score.

As repeated for the previous models, the K-means model is fitted only on the benign training split. The training data is first sorted chronologically by `window_start_ts` and `window_id` of which the first 80% is used to fit the K-means model,

while the remaining 20% is kept aside as a benign calibration subset for threshold selection. Features are scaled using `RobustScaler`, which is fitted only on the 80% model-fitting subset and then applied unchanged to the calibration and evaluation data.

After training, each calibration sample is assigned an anomaly score equal to its Euclidean distance to the nearest cluster centroid. As in the k-NN implementation described in Section 4.3.1, Euclidean distance is used after robust scaling of the selected features. The anomaly threshold is then selected as a tested quantile of these benign calibration distances. During evaluation, each validation or test window is scaled using the same scaler and assigned its nearest-centroid distance. A window is classified as anomalous when this distance exceeds the selected threshold.

Several values for the number of clusters are tested. Smaller values force the model to represent benign behavior using only a few broad traffic profiles, while larger values allow the model to capture more fine-grained modes of normal behavior. Similarly to the k-NN approach, different threshold quantiles are evaluated to study the trade-off between detecting malicious windows and limiting false positives on benign traffic. The implementation uses the `MiniBatchKMeans` class from `scikit-learn` [24].

4.4 Experimental Configuration

With the defined models, there are still several design choices that can influence their detection performance. These include the feature set, aggregation window length, anomaly threshold, and model-specific parameters such as the number of nearest neighbors, number of K-means clusters, autoencoder architecture, and Isolation Forest contamination value. In order to meaningfully select the model that would best perform for reconnaissance detection on conntrack-derived OT traffic, a grid of parameter combinations was evaluated.

The sweeps are organized around four main dimensions. First, multiple aggregation window sizes are evaluated in order to study how temporal granularity affects detection. Shorter windows may capture short scanning bursts more precisely, while longer windows may smooth traffic behavior and make low-rate reconnaissance activity more visible. Second, several feature sets are tested, ranging from more general reconnaissance based sets, to more case specialized sets as discussed in Section 4.2. Third, each model is evaluated under multiple threshold settings. For distance-based models (such as K-NN or K-means), thresholds are selected as quantiles of the benign calibration score distribution, and for the autoencoder different Isolation Forest contamination values are tested. Finally, model-specific parameters are varied, such as k for k-NN, the number of clusters for K-means, and the hidden-layer dimensions of the autoencoder.

After exploring all these possibilities, the validation data is used to evaluate which configurations show the most promising results. The validation results are then used to refine the next sweeps by adjusting parameter ranges and adding or re-

Model family	Configuration dimension	Evaluated values
All models	Aggregation window	60 s, 5 min, 1 h, and 5 h (60,000, 300,000, 3,600,000, and 18,000,000 ms)
All models	Feature sets	Candidate feature sets described in Appendix A.2
Distance-based models	Threshold quantiles	0.95, 0.99, 0.999, 0.9999, 0.99999, 0.999999, and 0.9999999
Distance-based models	Number of clusters/neighbours	1, 3, 5, and 10. For k-NN, $k = 5$ was excluded from the final reduced sweep based on the validation results (see Section 4.4.1).
Autoencoder	Architecture	N-24-4-24-N, N-36-6-36-N, N-64-8-64-N, and N-64-16-64-N
Autoencoder	Isolation Forest contamination	0.1, 0.01, 0.001, 0.0001, 10^{-5}, 10^{-6}, and 10^{-7}

Table 4.4: Overview of the evaluated model configuration space.

moving feature sets. Finally, the validation data is used to select the most promising model and parameter combinations for evaluation on the test set.

4.4.1 K-NN resource constraints

Due to the higher computational cost of k-NN, the k-NN experiments were not swept as exhaustively as the K-Means and autoencoder-based configurations. In particular, evaluating k-NN requires comparing each validation or test sample against the stored benign reference set, which made a full sweep over all feature sets, window sizes and values of k impractical with the available resources. K-Means and the autoencoder-based model were first evaluated over the full planned configuration space using the validation split, including all candidate feature sets, aggregation window sizes and threshold settings. These validation results were then used to identify feature sets that performed well across multiple reconnaissance scenarios and were not tailored to a single attack type.

Based on these validation results, a smaller exploratory k-NN sweep was performed. This sweep still evaluated all aggregation window sizes and threshold quantiles, but for a more reduced set of promising feature sets. Its purpose was twofold: first, to verify whether the selected feature sets also worked well for the distance-based k-NN model, and second, to compare the effect of different values of k.

The exploratory k-NN results showed that $k = 5$ did not provide a meaningful improvement over the other tested values. In the few cases where $k = 5$ obtained the highest score, the absolute detection performance remained low and was therefore not practically relevant. Consequently, $k = 5$ was excluded from the final k-NN sweep. The final k-NN evaluation was then performed on the selected top three feature sets based on all previous results, using all aggregation window sizes and all threshold quantiles for the remaining values of k.

4.5 Evaluation Metrics

The models are evaluated using both window-level and range-level metrics. Window-level F1 treats each window independently which can be limiting for reconnaissance detection, because an attack is usually not a single isolated window, but an activity that unfolds over a continuous time interval. A detector that raises at least one alarm during such an interval may already be useful from an operational perspective, even if it does not flag every malicious window. Similarly, a model may obtain a good window level F1-score while producing several separate false alarm intervals, which would be less desirable for an operator. This issue is also discussed by Fung et al. [12] who show that point-wise F1 can give a misleading impression of anomaly detection performance in ICS time-series settings, and argue that range-based metrics better capture operational objectives such as reducing false alarms and detecting attack intervals.

Window-level precision, recall and F1-score are computed from the number of true positives, false positives and false negatives over all evaluated windows. A true positive (TP) is a window that overlaps with an attack interval and is predicted as anomalous. A false positive (FP) is a benign window that is incorrectly predicted as anomalous, while a false negative (FN) is an attack window that is not detected. These metrics provide a more fine grained view and are useful for comparing how precisely each model identifies anomalous windows.

For the discussed range based precision, recall and F1-score, the true labels and model predictions are first processed separately for each source host. Consecutive positive windows are grouped into temporal ranges after sorting by timestamp. A true attack range is counted as detected when at least one predicted anomalous range overlaps with it. Predicted anomalous ranges that do not overlap with any true attack range are counted as range-level false positives, while true attack ranges without any overlapping prediction are counted as range-level false negatives. Hence, the range-based metric used here therefore follows an event-detection interpretation. It does not require every malicious window inside an attack interval to be classified correctly.

Window-level precision, recall and F1-score are defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{4.1}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4.2}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.3}$$

where TP, FP, and FN are counted over individual source-host windows.

The same precision, recall and F1-score definitions are then applied at the range level:

$$\text{Range Precision} = \frac{TP_{\text{range}}}{TP_{\text{range}} + FP_{\text{range}}} \tag{4.4}$$

$$\text{Range Recall} = \frac{TP_{\text{range}}}{TP_{\text{range}} + FN_{\text{range}}} \tag{4.5}$$

$$F_{1,\text{range}} = 2 \cdot \frac{\text{Range Precision} \cdot \text{Range Recall}}{\text{Range Precision} + \text{Range Recall}} \tag{4.6}$$

A predicted range $P = [p_s, p_e]$ overlaps a true attack range $T = [t_s, t_e]$ when:

$$p_s \leq t_e \wedge p_e \geq t_s.$$

Both metrics are reported because they provide complementary views of the same predictions. Window-level F1 evaluates each source-host window separately, while range-level F1 summarizes consecutive detections into temporal alarm ranges. Differences between them are therefore interpreted as different perspectives on model performance, rather than as contradictory results.

Chapter 5

Results

5.1 Evaluation Overview

is this too much repetition?

This chapter presents the experimental results of the evaluated anomaly detection models on the constructed reconnaissance detection datasets. The goal of the evaluation is to determine to what extent unsupervised models trained only on benign conntrack-derived traffic can detect injected reconnaissance activity in real OT background traffic. As described in Section 4.4, the configurations vary in feature set, aggregation window length, anomaly threshold, and model-specific parameters. For k-NN, the search space is more limited than for K-Means and the autoencoder because of its higher computational cost. The k-NN results are therefore interpreted as a targeted evaluation of validation-selected feature sets rather than as a full sweep over all candidate feature sets.

The experiments are evaluated in two stages. First, the validation split is used to compare configurations and identify promising model, feature-set and parameter combinations. These validation results are used for model selection and for understanding the effect of design choices such as window size and feature selection. The selected configurations are then evaluated on the test split, which is kept separate from the selection process and is used to assess how well the chosen configurations generalize to another period of network traffic.

The results are reported using both window-level and range-level metrics. Window-level metrics treat each source-host window as an individual prediction and therefore measure how accurately the model classifies separate time windows. Range-level metrics instead group consecutive anomalous windows into alarm ranges and compare these ranges with the true attack intervals. This provides a more operational view of the results, since reconnaissance activity usually unfolds over a period of time rather than in a single isolated window.

Both views are used throughout the discussion. Window-level F1 indicates how precise the model is at the individual-window level, while range-level F1 indicates whether complete reconnaissance scenarios are detected as temporal events and how many separate false alarm ranges are produced. A model may therefore have a

moderate window-level score while still detecting the relevant attack interval, or a strong window-level score while producing fragmented alarms.

Unless stated otherwise, the tables in this chapter report configurations selected according to validation performance, followed by their corresponding test-set performance. Differences between validation and test results are interpreted as an indication of robustness and sensitivity to the underlying benign traffic period.

5.2 Validation-Based Feature Set Selection

Before comparing the final model configurations on the test split, the validation results were used to identify the most promising feature sets. This step was especially important because the full feature-set search could not be repeated exhaustively for k-NN due to its higher computational cost, as discussed in Section 4.4.1. The selection was therefore mainly based on the full K-Means and autoencoder validation sweeps, which covered the complete planned feature-set space. The preliminary k-NN validation runs were then used as an additional check to verify whether the selected feature sets also performed well as well as the rejected ones preforming worse.

Feature set	#configs	Mean RF1	Med. RF1	Mean F1	Med. F1	Score
`csv_golden_minimal`	224	0.422	0.466	0.407	0.442	0.414
`csv_omni_core_clean`	224	0.432	0.500	0.382	0.459	0.407
`csv_low_slow_clean`	224	0.234	0.143	0.325	0.240	0.280
`csv_T2_stealth_scan`	224	0.253	0.100	0.266	0.076	0.259
`csv_recon_core_clean_temporal`	224	0.161	0.030	0.192	0.076	0.177
`csv_T3_T4_T5_loud_scan`	224	0.227	0.050	0.090	0.013	0.159
`csv_recon_core_clean`	224	0.135	0.025	0.130	0.037	0.133
`csv_horizontal_scan_clean`	112	0.185	0.019	0.046	0.009	0.115
`csv_ablation_entropy_only`	224	0.091	0.043	0.094	0.018	0.092
`csv_ablation_asymmetry_only`	112	0.000	0.000	0.000	0.000	0.000

Table 5.1: Validation-based feature set ranking. RF1 denotes range-level F1, and #configs is the number of validation configurations included for each feature set. The selection score is computed as the average of mean validation RF1 and mean validation window-level F1. Highlighted rows indicate the feature sets selected for the final cross-model comparison and targeted k-NN evaluation.

The goal of this selection was not only to find the highest individual validation score, but to identify feature sets that performed well across multiple reconnaissance scenarios. Feature sets that were designed for one specific attack type were therefore interpreted with caution, even when they achieved strong results on that specific attack. Instead, preference was given to feature sets with broader coverage across the

evaluated attacks. To summarize the validation results across feature sets, a compact selection score is used. This score gives equal weight to the mean validation range F1 and the mean validation window-level F1. Table 5.1 shows the resulting ranking. The two strongest feature sets are `csv_golden_minimal` and `csv_omni_core_clean`, which both achieve the highest combined validation scores. `csv_low_slow_clean` is also retained because it is the next strongest general-purpose feature set and is relevant for slower reconnaissance behaviour. Although `csv_T2_stealth_scan` achieves competitive scores in some cases, it is more closely tied to a specific attack type and is therefore not selected.

The three highest-scoring general feature sets thus are `csv_golden_minimal`, `csv_omni_core_clean`, and `csv_low_slow_clean`. These are therefore retained for the final cross-model comparison and for the reduced k-NN evaluation. The median values show that the two strongest feature sets are not only driven by a small number of high-performing configurations, but also perform more consistently across the evaluated validation configurations.

5.2.1 ADAM

In addition to the feature sets constructed specifically for reconnaissance detection, an ADAM-inspired feature set was also evaluated. As introduced in Section 2.5, this feature set was included because ADAM uses entropy-based traffic characteristics for anomaly detection in cyber-physical systems, and therefore provides a useful comparison point for evaluating whether a more compact entropy-oriented representation is sufficient for the present conntrack-based reconnaissance detection task.

The ADAM-inspired feature set was evaluated using the same validation procedure as the other candidate feature sets. However, its validation performance was not competitive and hence dropped at an early stage, as shown in Table **??**

FIXXXXXXXXX TABLEEEEEEEEEEEEEEE

These results suggest that the ADAM-inspired representation does not capture enough of the reconnaissance-specific behaviour present in the constructed dataset. In particular, the strongest selected feature sets combine several types of indicators, including destination spread, short-flow behaviour, failed or asymmetric communication, and temporal aggregation. The ADAM-inspired feature set is more limited in this respect, which may explain its weaker performance on the evaluated reconnaissance scenarios.

5.3 Overall Model Performance

5.4 Influence of Window Size

5.5 Influence of Feature Set

5.6 Detection Performance per Reconnaissance Scenario

5.7 Validation–Test Generalization

5.8 Operational Discussion

5.9 Limitations

Chapter 6

Conclusion

The final chapter contains the overall conclusion. It also contains suggestions for future work and industrial applications.

Appendices

Appendix A

The First Appendix

A.1 Feature Descriptions

Table A.1: Description of extracted window-level features.

Feature	Description
ip_src	Source host for which the window-level feature vector is constructed.
window_id	Integer identifier of the time window.
window_start_ts	Start timestamp of the time window in milliseconds.
flow_count	Number of unidirectional flows initiated by the source host in the window.
sent_bytes	Total number of bytes sent by the source host in the window.
sent_packets	Total number of packets sent by the source host in the window.
received_bytes	Total number of bytes received by the source host in the window.
received_packets	Total number of packets received by the source host in the window.
mean_duration_s	Mean flow duration in seconds for flows initiated by the source host.
std_duration_s	Standard deviation of flow duration in seconds.
mean_bytes_per_packet	Mean bytes-per-packet value across flows in the window.
std_bytes_per_packet	Standard deviation of bytes-per-packet values across flows.
bytes_per_flow	Average number of sent bytes per flow.
packets_per_flow	Average number of sent packets per flow.

Feature	Description
unique_dst_ip	Number of distinct destination IP addresses contacted by the source host.
unique_dst_port	Number of distinct destination ports contacted by the source host.
unique_src_port	Number of distinct source ports used by the source host.
dst_port_per_dst_ip_ratio	Ratio between distinct destination ports and distinct destination IP addresses.
outbound_ratio	Fraction of flows in the window for which the host is the originator.
reply_packet_ratio	Ratio of received packets to sent packets for the source host.
reply_byte_ratio	Ratio of received bytes to sent bytes for the source host.
no_reply_packet_score	Packet-based score indicating lack of reply traffic, computed as one minus the clipped reply packet ratio.
no_reply_byte_score	Byte-based score indicating lack of reply traffic, computed as one minus the clipped reply byte ratio.
is_pure_upload_like	Binary indicator set when almost all flows are outgoing.
is_pure_download_like	Binary indicator set when almost all flows are incoming.
l4_proto_entropy	Shannon entropy of transport-layer protocol usage for the source host in the window.
protocol_entropy	Alias for l4_proto_entropy.
ip_src_entropy_window	Window-level Shannon entropy of source IP activity across all hosts.
ip_dst_entropy	Shannon entropy of destination IP addresses contacted by the source host.
port_src_entropy	Shannon entropy of source ports used by the source host.
port_dst_entropy	Shannon entropy of destination ports contacted by the source host.
tcp_src_port_entropy	Shannon entropy of TCP source ports used by the source host.
tcp_dst_port_entropy	Shannon entropy of TCP destination ports contacted by the source host.
udp_src_port_entropy	Shannon entropy of UDP source ports used by the source host.
udp_dst_port_entropy	Shannon entropy of UDP destination ports contacted by the source host.

Feature	Description
tcp_flow_count	Number of TCP flows initiated by the source host in the window.
tcp_short_flow_count	Number of TCP flows with at most three packets.
tcp_short_flow_rate	Fraction of TCP flows that are short TCP flows.
flow_count_delta1	Difference in flow_count compared to the previous window for the same source host.
flow_count_mean3	Rolling mean of flow_count over the current and two previous windows for the same source host.
unique_dst_ip_delta1	Difference in unique_dst_ip compared to the previous window for the same source host.
unique_dst_ip_mean3	Rolling mean of unique_dst_ip over the current and two previous windows.
unique_dst_port_delta1	Difference in unique_dst_port compared to the previous window for the same source host.
unique_dst_port_mean3	Rolling mean of unique_dst_port over the current and two previous windows.
tcp_short_flow_rate_delta1	Difference in tcp_short_flow_rate compared to the previous window for the same source host.
tcp_short_flow_rate_mean3	Rolling mean of tcp_short_flow_rate over the current and two previous windows.
no_reply_packet_score_delta1	Difference in no_reply_packet_score compared to the previous window for the same source host.
no_reply_packet_score_mean3	Rolling mean of no_reply_packet_score over the current and two previous windows.
ip_dst_entropy_delta1	Difference in ip_dst_entropy compared to the previous window for the same source host.
ip_dst_entropy_mean3	Rolling mean of ip_dst_entropy over the current and two previous windows.
port_dst_entropy_delta1	Difference in port_dst_entropy compared to the previous window for the same source host.
port_dst_entropy_mean3	Rolling mean of port_dst_entropy over the current and two previous windows.
outbound_ratio_delta1	Difference in outbound_ratio compared to the previous window for the same source host.
outbound_ratio_mean3	Rolling mean of outbound_ratio over the current and two previous windows.

A.2 Feature Sets

Table A.2: Candidate feature sets used in the model evaluation.

Feature set	Included features
csv_omni_core_clean	tcp_short_flow_count, tcp_short_flow_rate_mean3, ip_dst_entropy_mean3, unique_dst_ip_mean3, no_reply_packet_score_mean3, outbound_ratio.
csv_golden_minimal	ip_dst_entropy_mean3, tcp_short_flow_rate_mean3, tcp_short_flow_count, no_reply_packet_score_mean3.
csv_recon_core_clean	ip_dst_entropy, tcp_short_flow_rate, unique_dst_ip, no_reply_packet_score, outbound_ratio.
csv_recon_core_clean_temporal	ip_dst_entropy, ip_dst_entropy_mean3, tcp_short_flow_rate, unique_dst_ip, no_reply_packet_score, outbound_ratio.
csv_T2_stealth_scan	ip_dst_entropy_mean3, tcp_short_flow_rate_mean3, unique_dst_ip, outbound_ratio_mean3.
csv_T3_T4_T5_loud_scan	tcp_short_flow_count, flow_count, unique_dst_ip_mean3, tcp_flow_count.
csv_low_slow_clean	ip_dst_entropy_mean3, tcp_short_flow_rate_mean3, unique_dst_ip_mean3, no_reply_packet_score_mean3, outbound_ratio_mean3.
csv_horizontal_scan_clean	unique_dst_ip, ip_dst_entropy, flow_count.
csv_adam_entropy	ip_src_entropy_window, ip_dst_entropy, protocol_entropy, tcp_src_port_entropy, tcp_dst_port_entropy, udp_src_port_entropy, udp_dst_port_entropy.
csv_ablation_entropy_only	ip_dst_entropy_mean3, port_dst_entropy_mean3, tcp_src_port_entropy, l4_proto_entropy.
csv_ablation_asymmetry_only	no_reply_packet_score_mean3, no_reply_byte_score, outbound_ratio, reply_packet_ratio.

Bibliography

[1] D. Abshari and M. Sridhar. A survey of anomaly detection in cyber-physical systems.

[2] D. Adams. *The Hitchhiker's Guide to the Galaxy*. Del Rey (reprint), 1995. ISBN-13: 978-0345391803.

[3] T. Behdadnia, K. Thoelen, and G. Deconinck. Early-stage anomaly detection in IT-OT power grid communication networks using wavelet transform and hybrid deep learning. 16(4):3305–3322.

[4] O. Cabana, A. M. Youssef, M. Debbabi, B. Lebel, M. Kassouf, and B. L. Agba. Detecting, fingerprinting and tracking reconnaissance campaigns targeting industrial control systems. In R. Perdisci, C. Maurice, G. Giacinto, and M. Almgren, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 89–108. Springer International Publishing.

[5] T. Cai, T. Jia, S. Adepu, Y. Li, and Z. Yang. ADAM: An adaptive DDoS attack mitigation scheme in software-defined cyber-physical system. 19(6):7802–7813.

[6] M. Catillo, A. Pecchia, and U. Villano. CPS-GUARD: Intrusion detection for cyber-physical systems and IoT devices using outlier-aware deep autoencoders. 129:103210.

[7] G. Dewaele, Y. Himura, P. Borgnat, K. Fukuda, P. Abry, O. Michel, R. Fontugne, K. Cho, and H. Esaki. Unsupervised host behavior classification from connection patterns. 20(5):317–337.

[8] J. Ding, C. Lu, and B. Li. A data-driven based security situational awareness framework for power systems. 94(11):1159–1168.

[9] djformby and S. Bryce. Grficsv3: Open source ot security lab. https://github.com/Fortiphyd/GRFICSv3, 2026. Accessed: 2026-05-04.

[10] J. Dromard, G. Roudière, and P. Owezarski. Online and scalable unsupervised network anomaly detection method. 14(1):34–47.

[11] C. Fung, S. Srinarasi, K. Lucas, H. B. Phee, and L. Bauer. Perspectives from a comprehensive evaluation of reconstruction-based anomaly detection in industrial control systems. In V. Atluri, R. Di Pietro, C. D. Jensen, and W. Meng,

editors, *Computer Security – ESORICS 2022*, volume 13556, pages 493–513. Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science.

[12] C. Fung, S. Srinarasi, K. Lucas, H. B. Phee, and L. Bauer. Perspectives from a comprehensive evaluation of reconstruction-based anomaly detection in industrial control systems. In V. Atluri, R. Di Pietro, C. D. Jensen, and W. Meng, editors, *Computer Security – ESORICS 2022*, volume 13556, pages 493–513. Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science.

[13] A. Guard. Meet the AXS guard team & company.

[14] A. Guard. Product features -, 2026.

[15] N. Jeffrey, Q. Tan, and J. R. Villar. A hybrid methodology for anomaly detection in cyber–physical systems. 568:127068.

[16] S. Kapoor, S. Kumar, and H. Vardhan. Cyber security of OT networks: A tutorial and overview.

[17] P. Keshavamurthy and S. Kulkarni. Early detection of reconnaissance attacks on IoT devices by analyzing performance and traffic characteristics. In *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 187–193.

[18] D. Lakens. Calculating and reporting effect sizes to facilitate cumulative science: a practical primer for t-tests and ANOVAs. 4:863.

[19] J. Meira, R. Andrade, I. Praça, J. Carneiro, V. Bolón-Canedo, A. Alonso-Betanzos, and G. Marreiros. Performance evaluation of unsupervised techniques in cyber-attack anomaly detection. 11(11):4477–4489.

[20] P. Moriano, S. C. Hespeler, M. Li, and M. Mahbub. Adaptive anomaly detection for identifying attacks in cyber-physical systems: A systematic literature review. 58(9):283.

[21] P. Neira Ayuso. Netfilter's connection tracking system. *;login:*, 31(3):34–40, 2006.

[22] Nmap. Nmap reference guide.

[23] I. Ortega-Fernandez, M. Sestelo, J. C. Burguillo, and C. Piñón-Blanco. Network intrusion detection system for DDoS attacks in ICS using deep autoencoders. 30(6):5059–5075.

[24] F. Pedregosa, F. Pedregosa, G. Varoquaux, G. Varoquaux, N. Org, A. Gramfort, A. Gramfort, V. Michel, V. Michel, L. Fr, B. Thirion, B. Thirion, O. Grisel, O. Grisel, M. Blondel, P. Prettenhofer, P. Prettenhofer, R. Weiss, V. Dubourg, V. Dubourg, J. Vanderplas, A. Passos, A. Tp, and D. Cournapeau. Scikit-learn: Machine learning in python. 2011.

[25] N. Project. The netfilter.org "nftables" project. https://www.netfilter.org/projects/nftables/index.html, 2025.

[26] M. Ring, D. Landes, and A. Hotho. Detection of slow port scans in flow-based network traffic. 13(9):1–18.

[27] M. Sarhan, S. Layeghy, and M. Portmann. Towards a standard feature set for network intrusion detection system datasets. *Mobile Networks and Applications*, 27(1):357–370, Nov. 2021.

[28] Z. Wang, M. A. Umer, H. Zhang, N. u. Hassan, and C. M. Ahmed. Guardians of ICS: A comparative analysis of anomaly detection techniques. 11(1):e70037. _eprint: https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/cps2.70037.

[29] A. A. Zaidy. Comprehensive analysis and detection of IoT network attacks using recon host discovery traffic dataset. 2(1):18–24.